

MODELO DE DISEÑO: CREACIÓN DE LOS DIAGRAMAS DE CLASES DE DISEÑO

Iterar es humano, ser recursivo, divino.

Anónimo

Objetivos

- Crear los diagramas de clases de diseño (DCD).
- Identificar las clases, métodos y asociaciones que se van a mostrar en un DCD.

Introducción

Una vez terminados los diagramas de interacción para las realizaciones de los casos de uso de la iteración actual de la aplicación de PDV NuevaEra, es posible identificar la especificación de las clases software (e interfaces) que participan en la solución software, y añadirles detalles de diseño, como los métodos.

UML proporciona la notación para representar los detalles de diseño en los diagramas de clase; en este capítulo, vamos a estudiarla y a crear los DCD.

19.1. Cuando crear los DCD

Aunque esta presentación de los DCD *viene después de* la creación de los diagramas de interacción, en la práctica normalmente se crean en paralelo. Al comienzo del diseño se podrían esbozar muchas clases, nombres de métodos y relaciones aplicando los patrones para asignar responsabilidades, antes de la elaboración de los diagramas de interacción. Es posible y deseable elaborar algo de los diagramas de interacción, actualizar entonces los DCD, después extender los diagramas de interacción algo más, y así sucesivamente.

Estos diagramas de clases podrían utilizarse como una notación más gráfica alternativa a las tarjetas CRC para recoger las responsabilidades y colaboraciones.

19.2. Ejemplo de DCD

El DCD de la Figura 19.1 ilustra una definición software parcial de las clases *Registro* y *Venta*.

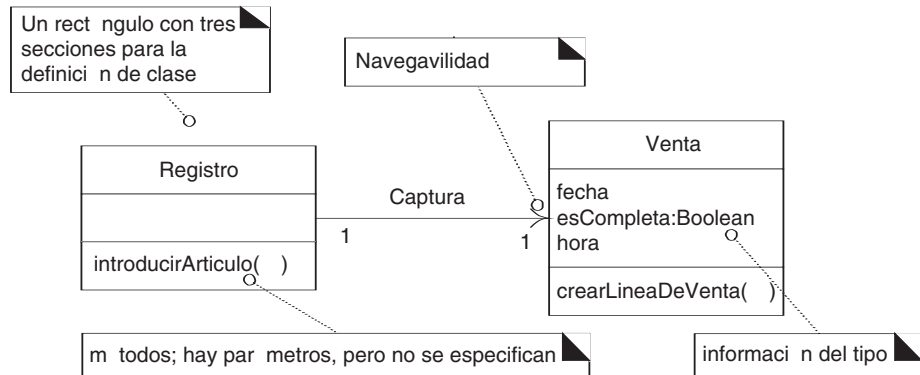


Figura 19.1. Ejemplo de diagrama de clases de diseño.

Además de las asociaciones y atributos básicos, el diagrama se amplía para representar, por ejemplo, los métodos de cada clase, información del tipo de los atributos, visibilidad de los atributos y navegación entre los objetos.

19.3. Terminología del DCD y el UP

Un **diagrama de clases de diseño** (DCD) representa las especificaciones de las clases e interfaces software (por ejemplo, las interfaces de Java) en una aplicación. Entre la información general encontramos:

- Clases, asociaciones y atributos.
- Interfaces, con sus operaciones y constantes.
- Métodos.
- Información acerca del tipo de los atributos.
- Navegabilidad.
- Dependencias.

A diferencia de las clases conceptuales del Modelo del Dominio, las clases de diseño de los DCD muestran las definiciones de las clases software en lugar de los conceptos del mundo real.

El UP no define de manera específica ningún artefacto denominado **diagrama de clases de diseño**. El UP define el Modelo de Diseño, que contiene varios tipos de diagramas, que incluye los diagramas de interacción, de paquetes, y los de clases. Los diagramas de clases del Modelo de Diseño del UP contienen **clases de diseño**. En términos del UP. De ahí que sea común hablar de **diagramas de clases de diseño** que es más corto que, e implica, **diagramas de clases** en el Modelo de Diseño.

19.4. Clases del Modelo de Dominio vs. clases del Modelo de Diseño

Seamos reiterativos, en el Modelo de Dominio del UP, una *Venta* no representa una definición software, sino que se trata de una abstracción de un concepto del mundo real acerca del cual estamos interesados en realizar una declaración. En cambio, los DCD expresan —para la aplicación software— la definición de las clases como componentes software. En estos diagramas, una *Venta* representa una clase software (ver Figura 19.2).

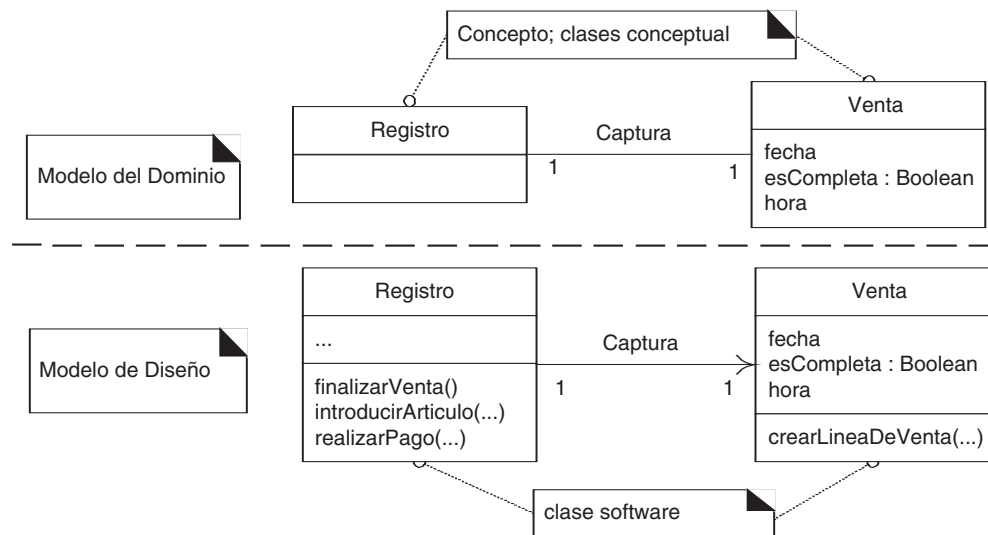


Figura 19.2. Clases del Modelo del Dominio vs. clases del Modelo de Diseño.

19.5. Creación de un DCD del PDV NuevaEra

Identificación y representación de las clases software

El primer paso en la creación de los DCD como parte del modelo de la solución es identificar aquellas clases que participan en la solución software. Se pueden encontrar examinando todos los diagramas de interacción y listando las clases que se mencionan.

Para la aplicación del PDV son:

Registro	Venta
CatalogoDeProductos	EspecificacionDelProducto
Tienda	LineaDeVenta
Pago	

El siguiente paso es dibujar un diagrama de clases para estas clases e incluir los atributos que se identificaron previamente en el Modelo del Dominio que también se utilizan en el diseño (ver Figura 19.3).

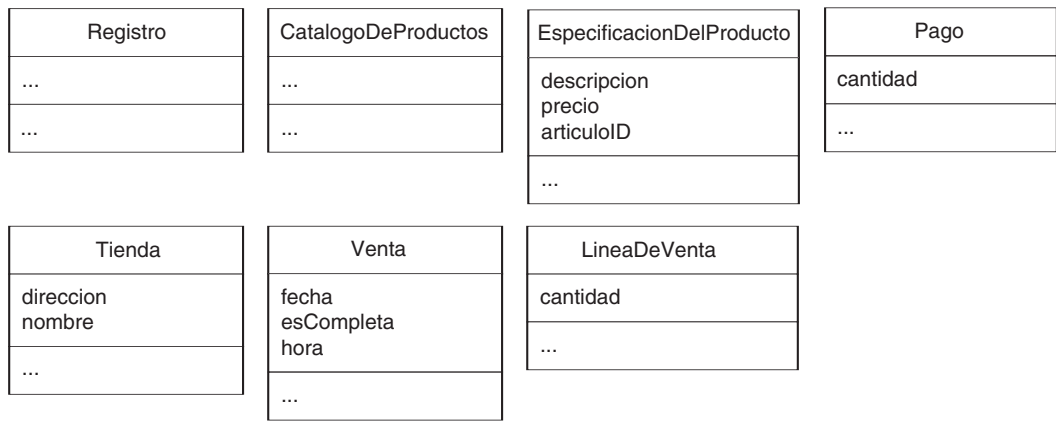


Figura 19.3. Clases software de la aplicación.

Nótese que algunos de los conceptos del Modelo del Dominio, como *Cajero*, no están presentes en el diseño. No es necesario —para la iteración actual— representarlos en el software. Sin embargo, en iteraciones posteriores, cuando se aborden nuevos requisitos y casos de uso, podrían formar parte del diseño. Por ejemplo, cuando se implementen los requisitos de seguridad y de inicio de sesión, es probable que sea relevante una clase software denominada *Cajero*.

A adir los nombres de los métodos

Se pueden identificar los nombres de los métodos analizando los diagramas de interacción. Por ejemplo, si se envía el mensaje *crearLineaDeVenta* a una instancia de la clase *Venta*, entonces la clase *Venta* debe definir un método *crearLineaDeVenta* (ver Figura 19.4).

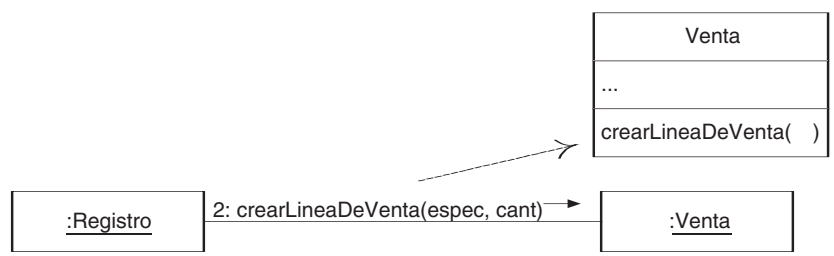


Figura 19.4. Nombres de los métodos a partir de los diagramas de interacción.

En general, el conjunto de todos los mensajes enviados a una clase X a lo largo de todos los diagramas de interacción indican la mayoría de los métodos que debe definir la clase X.

La inspección de todos los diagramas de interacción de la aplicación del PDV da lugar a la asignación de métodos que se muestra en la Figura 19.5.

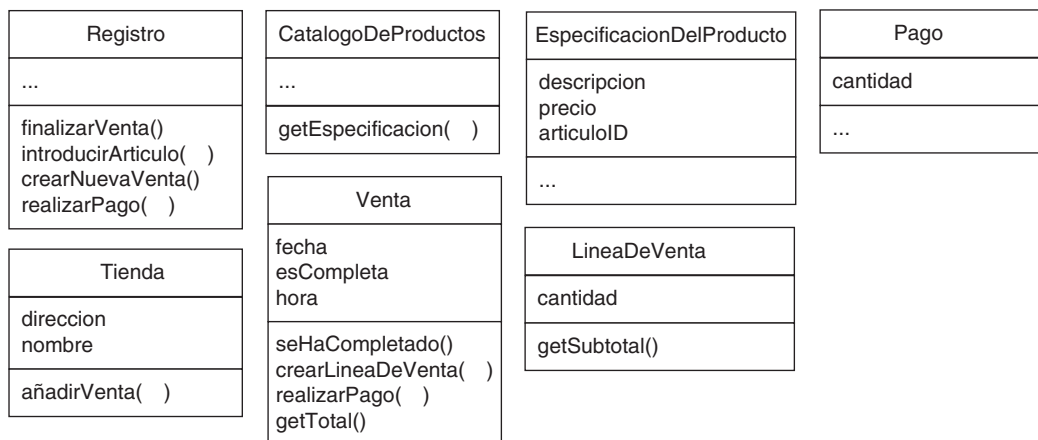


Figura 19.5. Métodos de la aplicación.

Cuestiones acerca de los nombres de los métodos

Se deben tener en cuenta las siguientes cuestiones especiales en relación con los nombres de los métodos:

- Interpretación del mensaje *create*.
- Descripción de los métodos de acceso.
- Interpretación de los mensajes a los multiobjetos.
- Sintaxis dependiente del lenguaje.

Nombres de los métodos—create

El mensaje *create* es una forma independiente del lenguaje posible en UML para indicar instanciación e inicialización. Al traducir el diseño a un lenguaje de programación orientado a objetos, se debe expresar en función de sus estilos para la instanciación e inicialización. No existe un método *create* ni en C++, ni en Java, ni en Smalltalk. Por ejemplo, en C++, implica la asignación automática, o asignación de almacenamiento libre con el operador *new*, seguido de una llamada al constructor. En Java, implica la invocación del operador *new*, seguido de una llamada al constructor.

Debido a las múltiples interpretaciones, y también debido a que la inicialización es una actividad muy común, es habitual omitir en el DCD los métodos relacionados con la creación y los constructores.

Nombres de los métodos—métodos de acceso

Los **métodos de acceso** recuperan (método de obtención, *accessor*) o establecen (método de cambio, *mutador*) el valor de los atributos. En algunos lenguajes (como Java) es un estilo común tener un accessor y un mutador para cada atributo, y declarar todos los

atributos como privados (para imponer la encapsulación de datos). Normalmente, se excluye la descripción de estos métodos en el diagrama de clases debido a que generan mucho ruido; para n atributos, hay $2n$ métodos sin interés. Por ejemplo, no se muestra, aunque está presente, el método *getPrecio* (o *precio*) de la *EspecificacionDelProducto*, porque se trata simplemente de un método de acceso.

Nombres de los métodos—multiobjetos

Un mensaje a un multiobjeto se interpreta como un mensaje al propio objeto contenedor/colección. Por ejemplo, el siguiente mensaje al multiobjeto *buscar* se debe interpretar como un mensaje al objeto contenedor/colección, como a un *Map* en Java, un *map* en C++, o un *Dictionary* en Smalltalk (ver Figura 19.6).

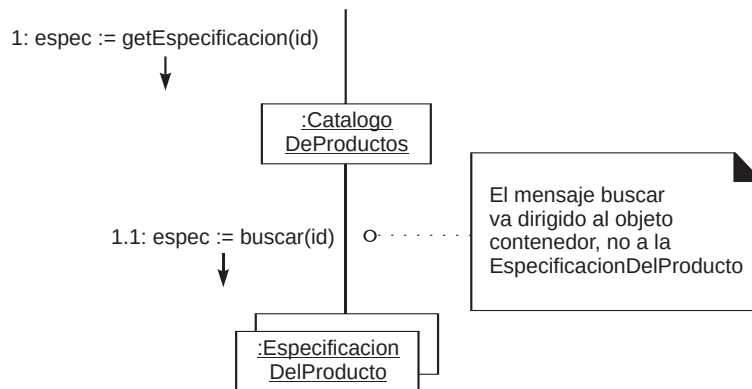


Figura 19.6. Mensaje a un multiobjeto.

Por tanto, el método *buscar* no forma parte de la clase *EspecificacionDelProducto* sino de la interfaz del multiobjeto. En consecuencia, no es correcto añadir el método *buscar* a la clase *EspecificacionDelProducto*.

Normalmente, estas interfaces o clases de contenedores/colecciones (como la interfaz *java.util.Map*) son elementos de las librerías predefinidas, y no es útil mostrar explícitamente estas clases en el DCD, porque añaden ruido, pero poca información nueva.

Nombres de los métodos—sintaxis dependiente del lenguaje

Algunos lenguajes, como Smalltalk, tienen una sintaxis que es muy diferente del formato UML básico de *nombreMetodo(listaParametros)*. Se recomienda que se utilice el formato UML básico, incluso si el lenguaje de implementación que se planea utilizar tiene una sintaxis diferente. Idealmente, la traducción debería tener lugar en el momento de la generación de código, en lugar de durante la creación de los diagramas de clase. Sin embargo, UML permite otra sintaxis para la especificación de los métodos.

Añadir más información sobre los tipos

Opcionalmente, todos los tipos de los atributos, parámetros de los métodos y los valores de retorno se podrían mostrar. La cuestión sobre si se muestra o no esta información se debe considerar en el siguiente contexto:

Se debería crear un DCD teniendo en cuenta los destinatarios.

Si se está creando en una herramienta CASE con generación automática de código, son necesarios todos los detalles y de modo exhaustivo.

Si se está creando para que lo lean los desarrolladores de software, los detalles exhaustivos de bajo nivel podrían afectar negativamente por el nivel de ruido.

Por ejemplo, ¿es necesario mostrar todos los parámetros y la información de sus tipos? Depende de lo obvia que sea la información para la audiencia a la que está destinada.

El diagrama de clases de diseño de la Figura 19.7 muestra más información sobre los tipos.

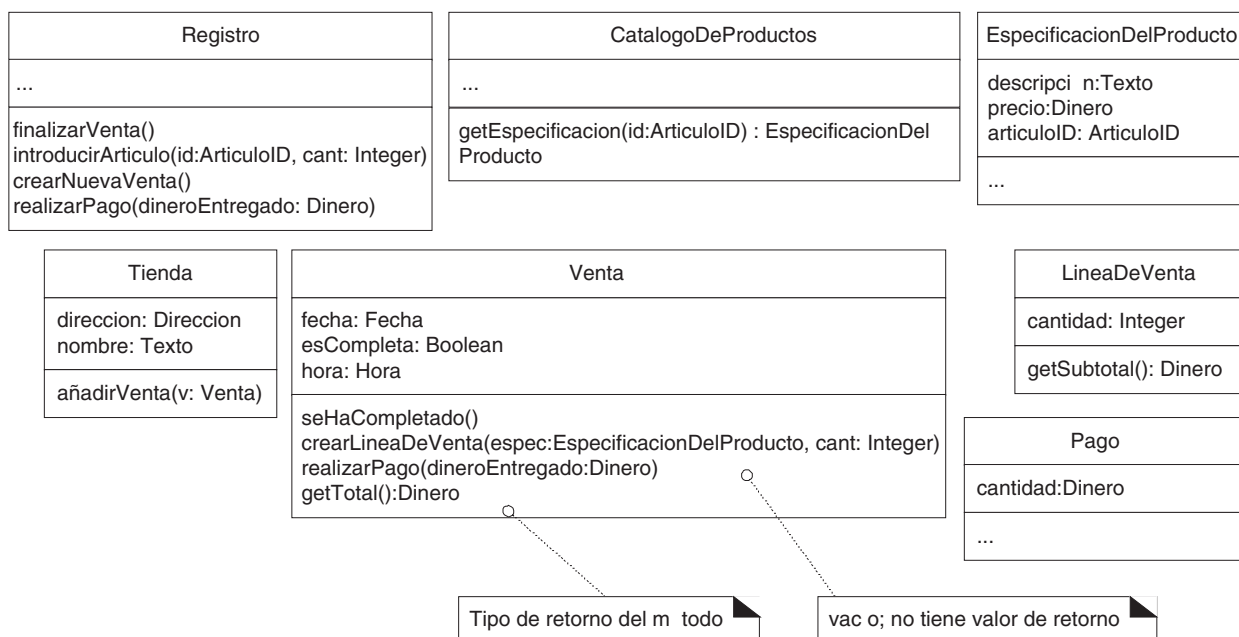


Figura 19.7. Añadir información acerca de los tipos.

Añadir asociaciones y navegabilidad

Cada extremo de asociación se denomina rol, y en los DCD el rol podría decorarse con una flecha de navegabilidad. La **navegabilidad** es una propiedad del rol que indica que es posible navegar unidireccionalmente a través de la asociación desde los objetos de la clase origen a la clase destino. La navegabilidad implica visibilidad —normalmente visibilidad de atributo (ver Figura 19.8)—.

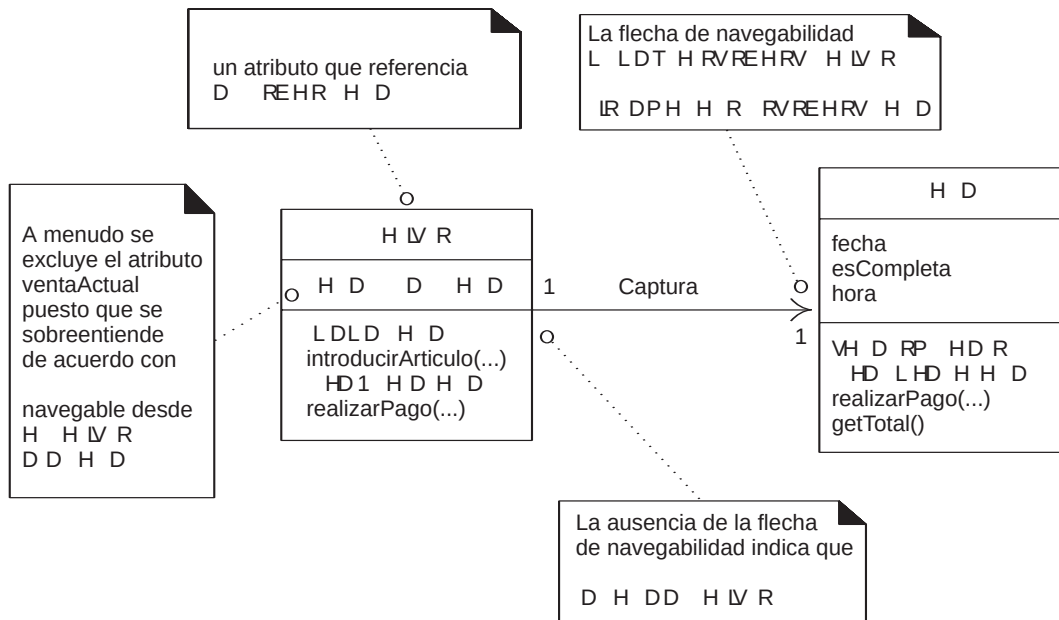


Figura 19.8. Representación de la navegabilidad o visibilidad de atributo.

La interpretación habitual de una asociación con una flecha de visibilidad es la visibilidad de atributo desde la clase origen hasta la clase destino. Durante la implementación en un lenguaje orientado a objetos por lo general se transforma en un atributo en la clase origen que hace referencia a una instancia de la clase destino. Por ejemplo, la clase *Registro* definirá un atributo que referencia a una instancia de *Venta*.

La mayoría, por no decir todas, las asociaciones en los DCD deberían adornarse con las flechas de navegabilidad necesarias.

En un DCD, se eligen las asociaciones de acuerdo a un criterio necesito-conocer orientado al software estricto —¿qué asociaciones se requieren para satisfacer la visibilidad y las necesidades de memoria actuales que se indican en los diagramas de interacción?—. Esto contrasta con las asociaciones en el Modelo del Dominio, que se podrían justificar por la intención de mejorar la comprensión del dominio del problema. Una vez más, vemos que existe una diferencia entre los objetivos del Modelo del Diseño y del Modelo del Dominio: uno es analítico, el otro una descripción de los componentes software.

La visibilidad y las asociaciones requeridas entre las clases se dan a conocer mediante los diagramas de interacción. A continuación, presentamos algunas situaciones comunes que sugieren la necesidad de definir una asociación con un adorno de visibilidad de A a B:

- A envía un mensaje a B.
- A crea una instancia de B.
- A necesita mantener una conexión a B.

Por ejemplo, a partir del diagrama de interacción de la Figura 19.9 que comienza con el mensaje *create* a la *Tienda*, y a partir del contexto más amplio de los otros diagramas

de interacción, se puede distinguir que probablemente la *Tienda* debería tener una conexión permanente con las instancias de *Registro* y el *CatalogoDeProductos* que crea. También es razonable que el *CatalogoDeProductos* necesite una conexión permanente con la colección de objetos *EspecificacionDelProducto* que crea. De hecho, muy a menudo el creador de otro objeto requiere una conexión permanente con él. Por tanto, las conexiones implícitas se presentarán como asociaciones en el diagrama de clases.

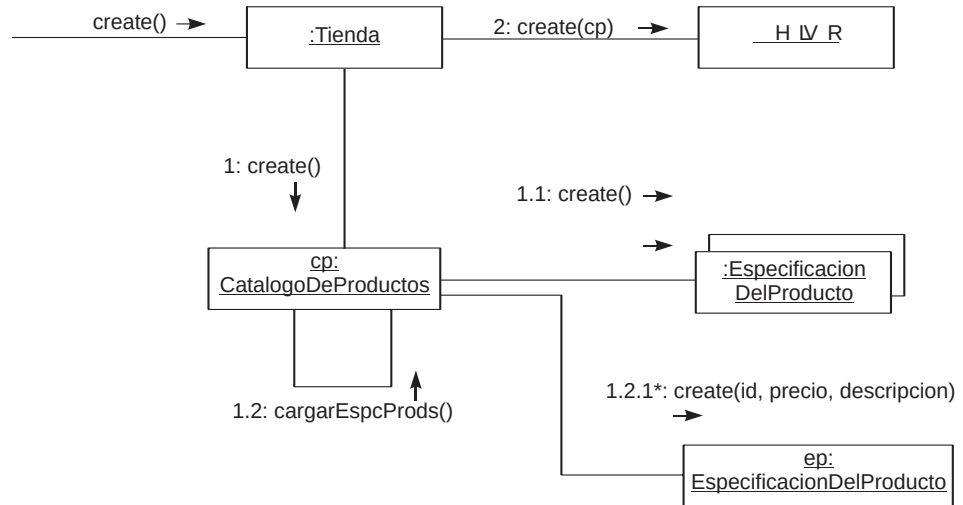


Figura 19.9. La navegabilidad se identifica a partir de los diagramas de interacción.

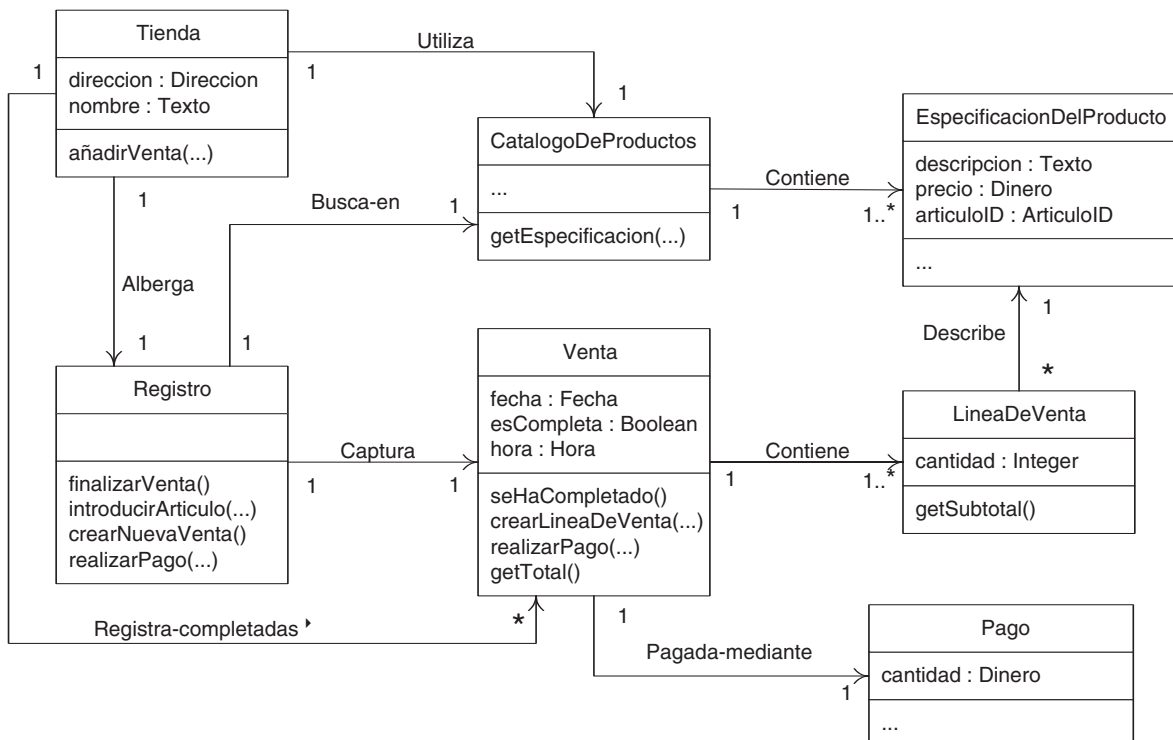


Figura 19.10. Asociaciones con adornos de navegabilidad.

Basado en el criterio anterior para las asociaciones y la navegabilidad, el análisis de todos los diagramas de interacción generados para la aplicación del PDV NuevaEra dará lugar a un diagrama de clases (ver Figura 19.10) con las siguientes asociaciones (se oculta la información exhaustiva sobre los tipos por claridad).

Nótese que éste no es exactamente el mismo conjunto de asociaciones que se generó para el diagrama de clases del Modelo del Dominio. Por ejemplo, en el modelo del dominio no existía la asociación *Busca-en* entre el *Registro* y el *CatalogoDeProductos* —no se pensó que fuera una relación importante y permanente en ese momento—. Pero durante la creación de los diagramas de interacción, se decidió que el objeto software *Registro* debería tener una conexión permanente con el *CatalogoDeProductos* software para buscar los objetos *EspecificacionDelProducto*.

Añadir las relaciones de dependencia

UML incluye una **relación de dependencia** general, que indica que un elemento (de cualquier tipo, como clases, casos de uso, etc.) tiene conocimiento de otro elemento. Se representa mediante una línea de flecha punteada. En los diagramas de clases la relación de dependencia es útil para describir la visibilidad entre clases que no es de atributo; en otras palabras, declaración de visibilidad de parámetro, local o global. En cambio, la vi-

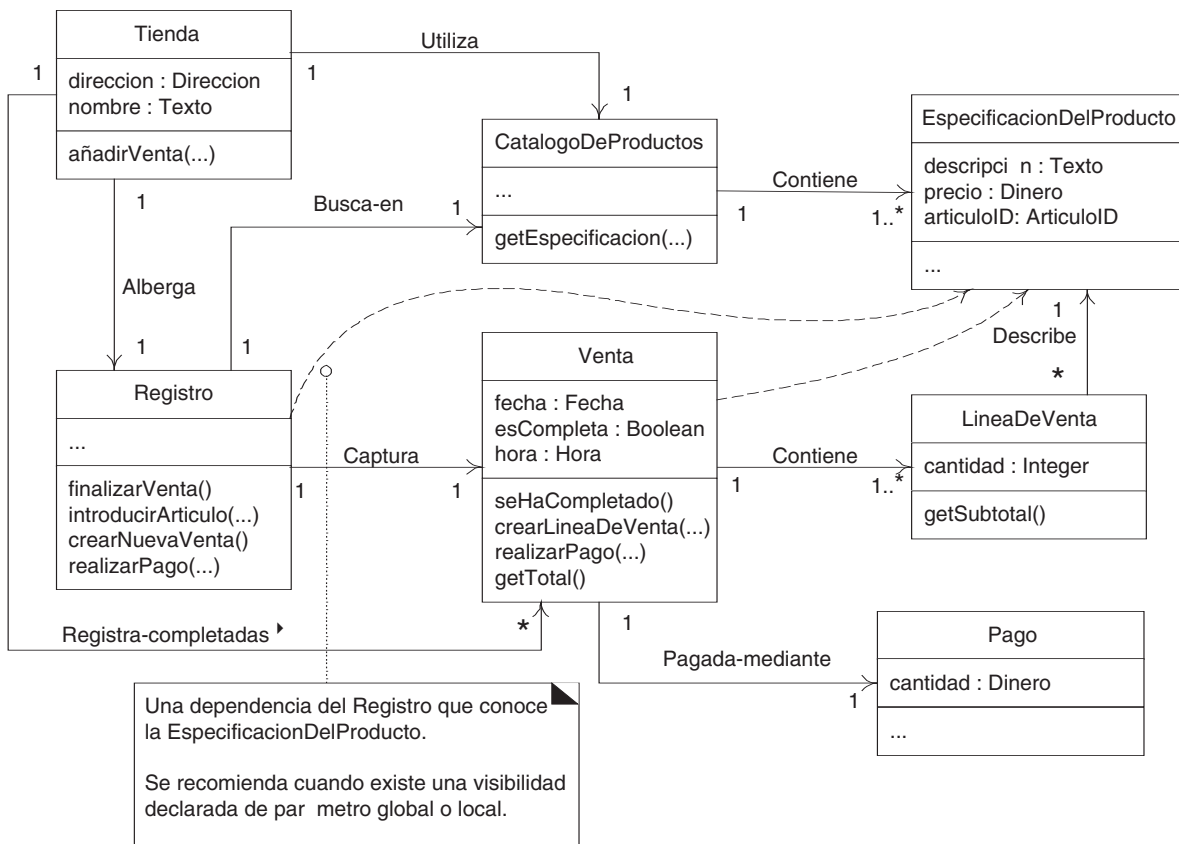


Figura 19.11. Relaciones de dependencia que indican una visibilidad que no es de atributo.

sibilidad de atributo simple se muestra mediante una línea de asociación ordinaria y una flecha de navegabilidad. Por ejemplo, el objeto software *Registro* recibe un objeto de retorno de tipo *EspecificacionDelProducto* a partir del mensaje de especificación que envía al *CatalogoDeProductos*. Por tanto, el *Registro* tiene una visibilidad a corto plazo, declarada localmente, de la *EspecificacionDelProducto*. Y una *Venta* recibe una *EspecificacionDelProducto* como parámetro en el mensaje *crearLineaDeVenta*; tiene visibilidad de parámetro de una especificación.

Estas visibilidades que no son de atributo podrían representarse con la línea de flecha punteada que indica una relación de dependencia (ver Figura 19.11). La curvatura de las líneas de dependencias no tiene ningún significado; se acomoda a la representación gráfica.

19.6. Notación para los detalles de los miembros

UML proporciona una notación variada para describir las características de los miembros de las clases e interfaces, como la visibilidad, valores iniciales, etcétera. En la Figura 19.12 se muestra un ejemplo.

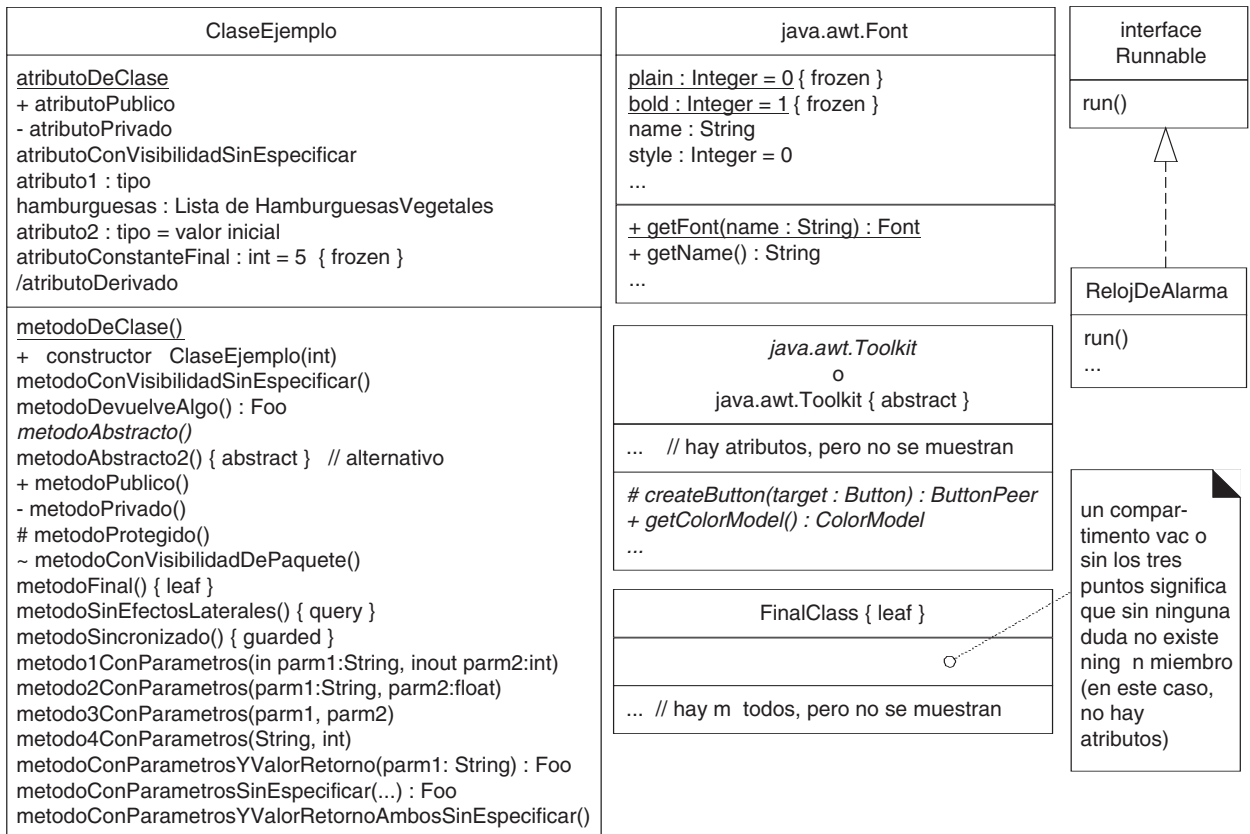


Figura 19.12. Detalles de la notación UML para algunos miembros de los diagramas de clases.

Visibilidad por defecto en UML

Si no se muestra explícitamente ningún marcador de visibilidad para un atributo o método, ¿cuál es el valor por defecto? Respuesta: no hay un valor por defecto. Si no se muestra nada, en UML significa sin especificar. Sin embargo, la convención común es asumir que los atributos son privados y los métodos públicos, a menos que se indique otra cosa.

La iteración actual del diagrama de clases de diseño del PDV NuevaEra (ver Figura 19.13) no tiene muchos detalles interesantes de los miembros; todos los atributos son privados y todos los métodos públicos.

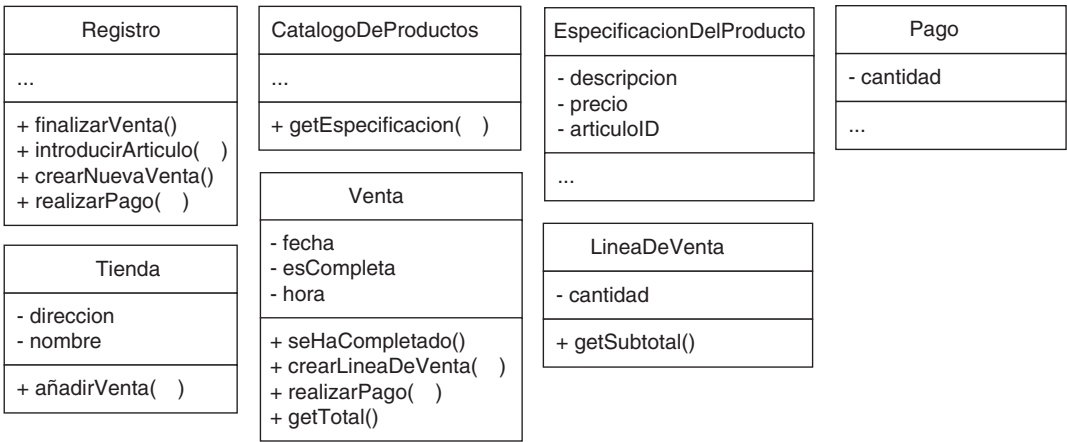


Figura 19.13. Detalles de los miembros del diagrama de clases del PDV.

Notación para el cuerpo de los métodos en los DCD (y los diagramas de interacción)

El cuerpo de un método se puede representar como se ilustra en la Figura 19.14 tanto en el DCD como en el diagrama de interacción.

19.7. DCD, dibujo y herramientas CASE

Las herramientas CASE pueden hacer ingeniería inversa (generar) de los DCD a partir del código fuente. En el Capítulo 35 sobre el dibujo y las herramientas CASE, se presentará una breve discusión sobre el contexto del proceso y la práctica de dibujar los DCD.

19.8. DCD en el UP

Los DCD forman parte de la realización de los casos de uso y, por tanto, miembros del Modelo de Diseño del UP.

Fases

Inicio: El Modelo de Diseño y los DCD normalmente no comenzarán hasta la elaboración porque comprenden decisiones de diseño, que son prematuras durante la fase de inicio.

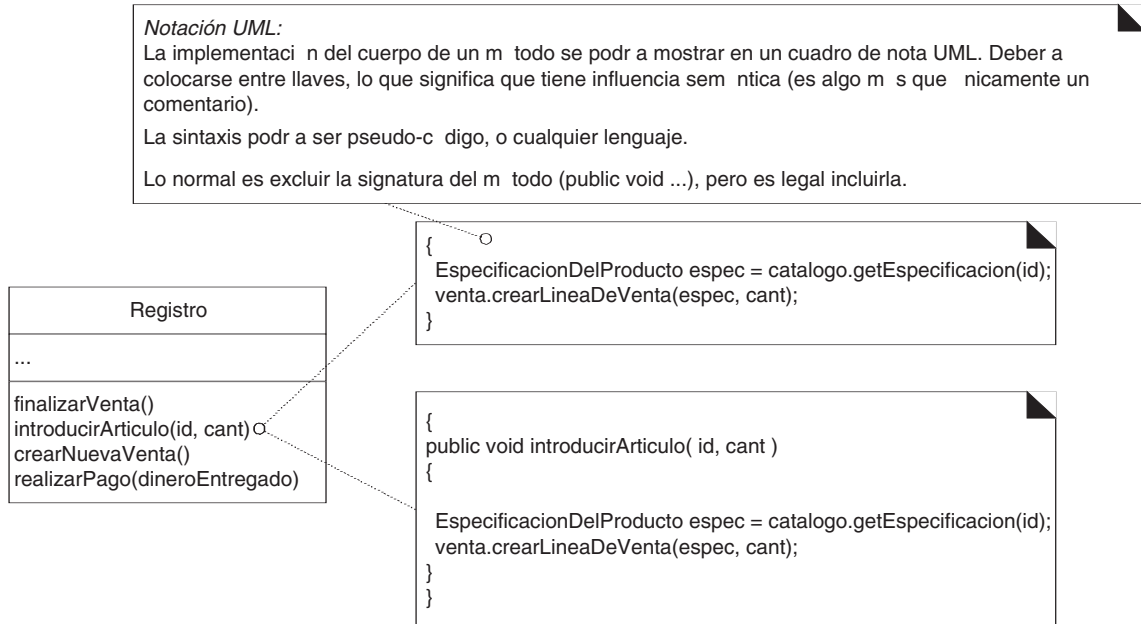


Figura 19.14. Notación para el cuerpo de los métodos.

Tabla 19.1. Muestra de los artefactos UP y evolución temporal. c-comenzar; r-refinar

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso (DSS) Visión Especificación Complementaria Glosario	c c c c	r r r r		
Diseño	Modelo de Diseño Documento de Arquitectura SW Modelo de Datos		c c c	r r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Elaboración: Durante esta fase, los DCD acompañarán a los diagramas de interacción de las realizaciones de los casos de uso; podrían crearse para las clases del diseño más significativas desde el punto de vista de la arquitectura.

Nótese que las herramientas CASE pueden hacer ingeniería inversa (generar) de los DCD a partir del código fuente. Es recomendable generar los DCD de manera regular a partir del código fuente, para visualizar la estructura estática del sistema.

Construcción: Los DCD se continuarán generando a partir del código fuente como apoyo a la visualización de la estructura estática del sistema.

19.9. Artefactos del UP

La Figura 19.15 muestra la influencia entre los artefactos destacando los DCD.

Muestra de las relaciones entre los artefactos del UP para Diagramas de Clases de Diseño

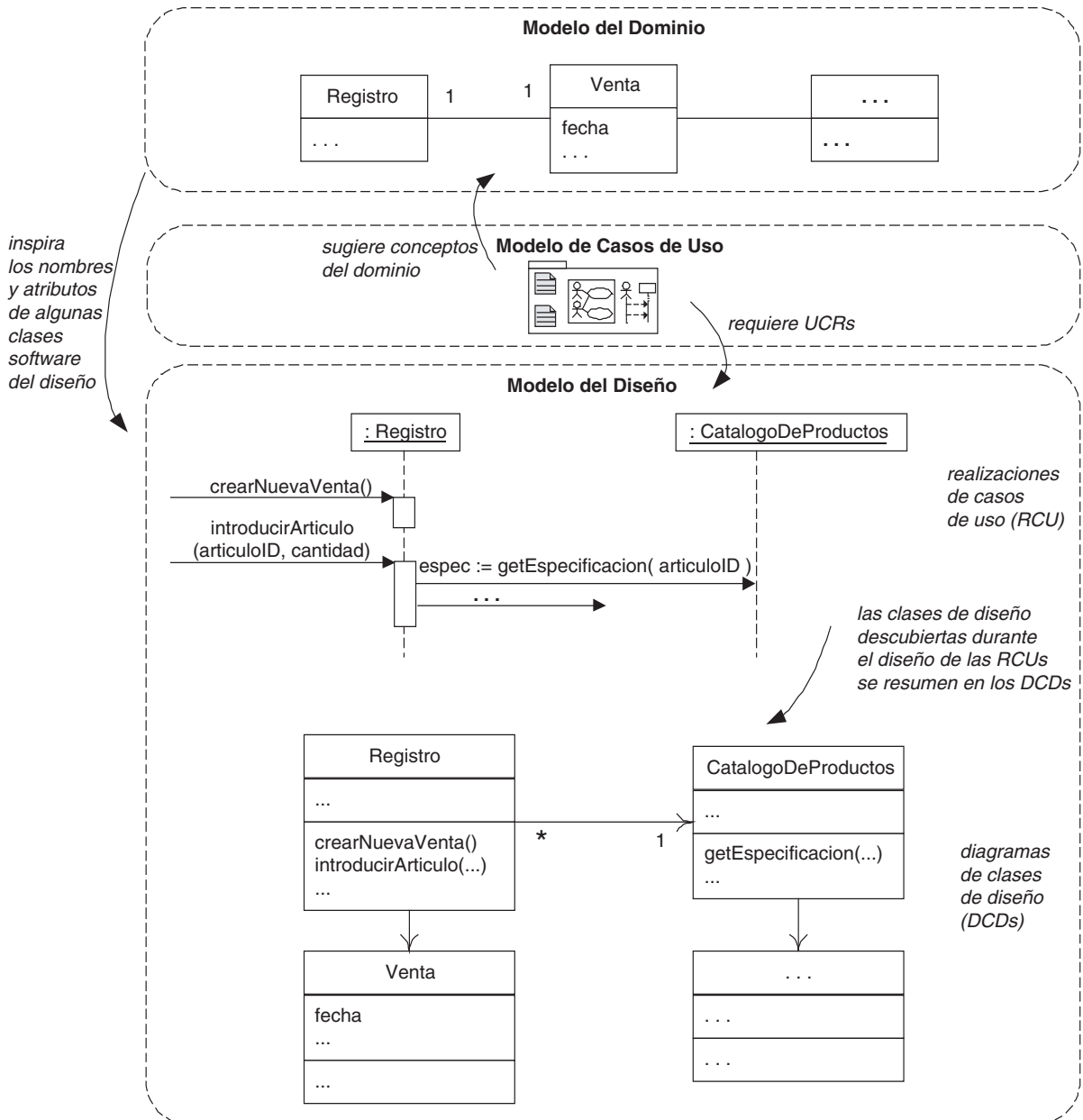


Figura 19.15. Muestra de la influencia entre los artefactos del UP.